

Scenario 3: Ambiguous Requirement, "Make it Enterprise-Ready"

Requirement

"Make it enterprise-ready."

Ambiguities Identified

This requirement is underspecified. "Enterprise-ready" could mean any of:

- 1 Security: authentication, authorization, audit logging, input sanitization.
- 2 Reliability: HA infrastructure, graceful degradation, circuit breakers.
- 3 Observability: distributed tracing, structured logging, alerting, dashboards.
- 4 Governance: custom domains, link ownership, per-link access control.
- 5 Compliance: data retention, GDPR right-to-erasure, PII handling.
- 6 Scalability: horizontal scaling, load balancing, CDN integration.
- 7 Developer experience: API docs, SDK, versioned APIs.
- 8 Operational: feature flags, configuration management, zero-downtime deploys.
- 9 Data quality: link expiry, cleanup, abuse prevention.

Without stakeholder input, prioritization is impossible. The interpretations below are what I chose for this assessment, with rationale for each.

Interpretation Decisions

- 1. Abuse prevention (rate limiting).** A public shortener without rate limiting gets used for spam almost immediately, and it's the most pressing "enterprise" concern for a brand-new service. Decision: IP-based rate limiting via Bucket4j (token bucket), 10 requests per minute per IP.
- 2. Link governance (custom aliases + expiry).** Enterprise users need branded links (`company.com/summer-sale`) and time-bounded campaigns. Decision: an optional custom alias (alphanumeric plus hyphen/underscore, max 50 chars), an optional `expiresAt`, and a 410 Gone response for expired links.
- 3. Data hygiene (cleanup job).** Expired URLs accumulate and waste storage over time. Decision: a scheduled job at 2 AM daily that deletes expired records.
- 4. Health and observability (Actuator).** Ops teams need health endpoints for load balancers and monitoring. Decision: Spring Boot Actuator with health, info, and metrics.
- 5. AI-powered URL safety check.** A public shortener can be weaponized for phishing or malware distribution, so enterprise deployments need content moderation on submitted URLs. It's also the most on-theme "enterprise" concern for an AI-focused role: using AI as a functional runtime component, not only a coding tool. Decision: integrate Claude (`claude-haiku-4-5`) as a real-time safety classifier on the creation path. A structured prompt asks the model to classify the URL as safe or unsafe; unsafe URLs are rejected with 422 and the model's reasoning is returned to the caller. It degrades gracefully too: no API key or a failed call means the URL is allowed through (fail-open), and the redirect path is never affected either way.

Deferred (out of scope for the prototype): authentication/authorization, which would need identity provider integration; distributed tracing, which needs Zipkin or Jaeger; CDN integration, which is an infrastructure concern rather than application code.

Task Decomposition

Custom aliases. `customAlias` field with a `UNIQUE` constraint; `existsByCustomAlias()`; validation in `WriteService`; `DuplicateAliasException` mapped to 409; a pattern-validated field on `ShortenRequest`; a frontend input on `Home`.

TTL/expiry. `expiresAt` field; `isExpired()` on the entity; expiry check in `ReadService` before redirecting; `UrlExpiredException` mapped to 410; an expiry picker on the frontend; an `Active/Expired` badge on `Manage`.

Rate limiting. `bucket4j-core` dependency; `RateLimitConfig` with a per-IP bucket map; `RateLimitExceededException` mapped to 429; the check runs in `UrlController.shorten()` before any processing.

Cleanup job. `findByExpiresAtBeforeAndExpiresAtIsNotNull()`; `ExpiredUrlCleanupService` with `@Scheduled(cron = "0 0 2 * * *")`; `@EnableScheduling` on the main application class.

Actuator. `spring-boot-starter-actuator`; `health/info/metrics` endpoints enabled in `application.yml`.

AI-powered URL safety check. `UrlSafetyService` calls the Claude API via `java.net.http.HttpClient`, parses a `{safe, reason}` JSON response, and strips the markdown code fences the model sometimes wraps its output in; `UrlNotSafeException` maps to 422; it's called in `WriteService.shorten()` before any other processing; degrades gracefully on a missing key or an API failure; `anthropic.api-key` reads from `ANTHROPIC_API_KEY`; Swagger UI comes from `springdoc-openapi-starter-webmvc-ui`.

Acceptance Criteria

Criterion	Verified by
POST with custom alias creates URL with that code	<code>WriteServiceTest.usesCustomAliasWhenProvided</code>
POST with duplicate alias returns 409	<code>WriteServiceTest.throwExceptionForDuplicateAlias</code>
Custom alias allows alphanumeric/hyphen/underscore	Bean Validation pattern check
Custom alias rejects spaces/special characters	Invalid request returns 400
Expired URL returns 410 on redirect	<code>ReadServiceTest.throwExpiredForExpiredUrl</code>
POST with <code>expiresAt</code> persists TTL	DB record has <code>expires_at</code> set
11th POST/minute from one IP returns 429	Manual test at limit 10
Cleanup job deletes expired URLs	Manual scheduler run, DB check
GET <code>/actuator/health</code> returns UP	<code>curl</code>
Expired URL shows "Expired" in Manage page	Frontend visual test
POST with phishing URL returns 422 + AI reason	Manual test with a fake credential-harvesting URL
POST with safe URL proceeds normally	Normal URL shortens successfully
POST with no API key, check skipped	Remove <code>ANTHROPIC_API_KEY</code> , URL shortens normally
POST when AI API times out, URL allowed through	Fail-open behavior verified in logs
Swagger UI shows all endpoints	GET <code>/swagger-ui.html</code>

Trade-offs Documented

Rate limit storage: in-memory vs. Redis. Chosen: in-memory `ConcurrentHashMap`. It resets on restart, and multiple instances don't share limits. The production alternative, `bucket4j-redis`, shares bucket state across instances but wasn't implemented here to avoid coupling rate limiting to Redis availability in the prototype.

Cleanup job: schedule vs. event-driven. Chosen: a cron job at 2 AM, with max 24h staleness before expired-but-not-yet-purged rows are deleted (they're never served in the meantime, see below). A production alternative would be Kafka delayed-expiry events for near-real-time cleanup, at added complexity.

Alias validation: application vs. database only. Chosen: both. A regex check at the API layer gives a friendly error, and the `UNIQUE` constraint is the race-condition safety net.

Expiry check: read-time vs. scheduled cleanup. Chosen: both. `isExpired()` is checked on every cache miss, and the cleanup job handles bulk deletion. Expired rows can sit in the DB for up to 24h, but they're never served to end users during that window.